

Distributed architectures

From one deployable to many: SOA, microservices, and the trade-offs in between.

Contents

01 The monolith

02 Going distributed

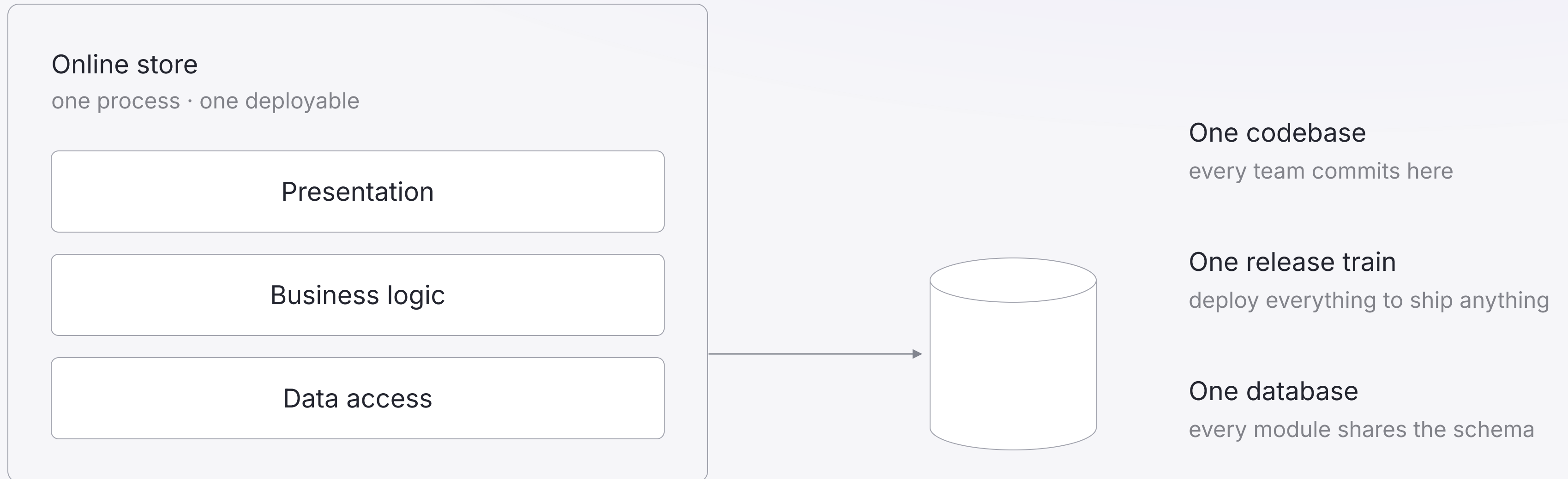
03 Benefits & risks

04 Choosing an architecture

01

— The monolith

Anatomy of a monolith



What monoliths do well

Simple to run

One artifact to build, test, deploy and monitor. A single stack trace tells the whole story of a failure.

Fast inside

Modules talk through function calls, not networks: no serialization, no timeouts, no partial failure between them.

Real transactions

One database means ACID transactions across the whole domain — reserve stock and take payment atomically.

Where monoliths strain

All-or-nothing scaling

The hot 5% of the system and the idle 95% scale together — you replicate the whole box to relieve one path.

One release train

Every team ships through the same pipeline; one bad commit or slow test suite blocks everyone behind it.

Eroding boundaries

Nothing stops a quick import across modules. Ten years in, everything touches everything — and nothing moves.

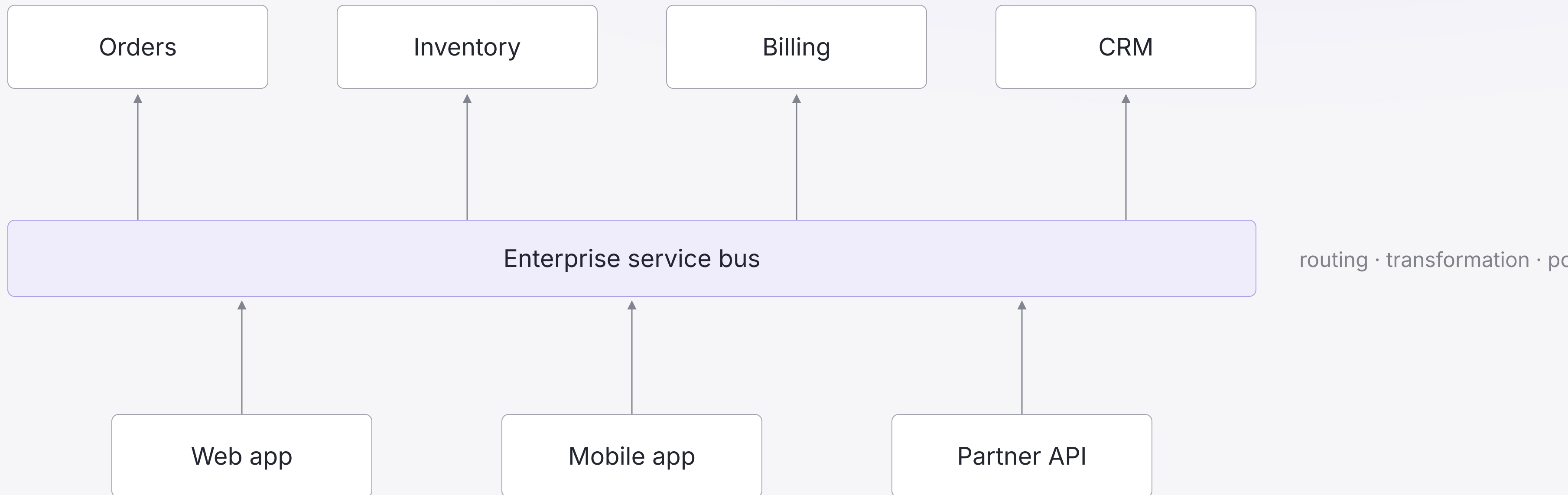
02

— Going distributed

The shape of a distributed system

- Independent components on different machines, cooperating over a network
- No shared memory — every interaction is a message that takes time
- Partial failure: one part can be down while the rest keeps running
- Independent lifecycles — each part is built, deployed and scaled on its own

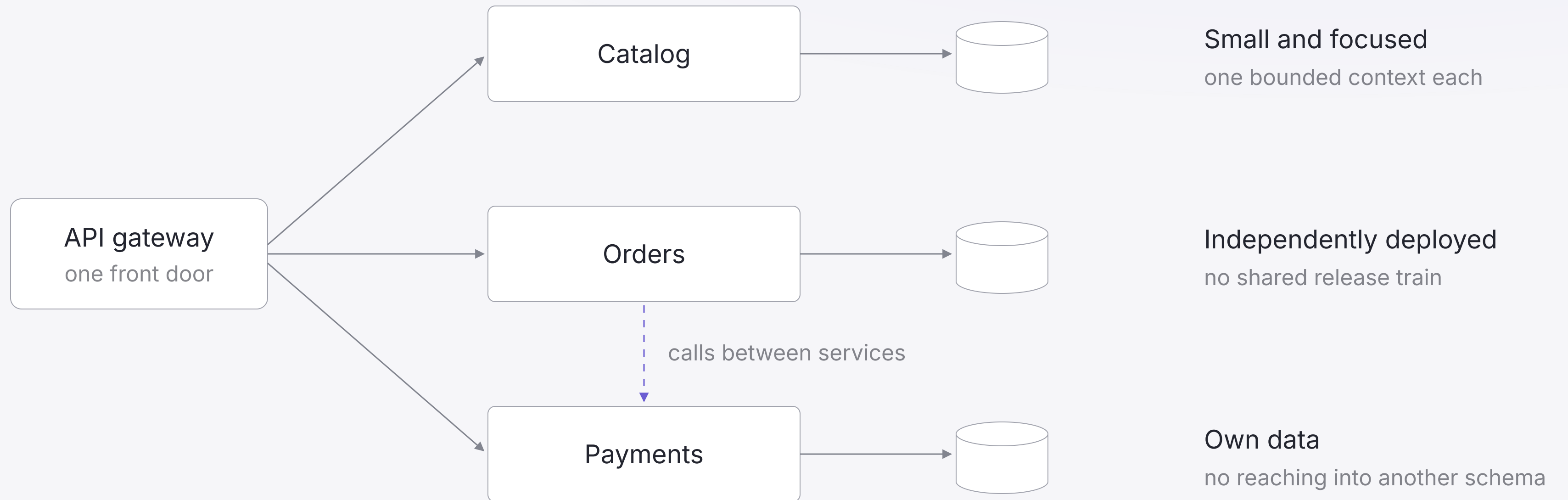
Service-oriented architecture



Every conversation passes through the bus — integration logic lives in the middleware, and the bus

becomes the thing everyone waits on.

Microservices



SOA vs microservices

DIMENSION	SOA	MICROSERVICES
Integration	Smart bus in the middle	Dumb pipes, smart endpoints
Scope	Enterprise-wide reuse	One bounded context per service
Data	Often shared databases	Database per service
Deployment	Coordinated releases	Independent deploys
Governance	Central standards body	Team autonomy

The eight fallacies

- The network is reliable
- Latency is zero
- Bandwidth is infinite
- The network is secure
- Topology doesn't change
- There is one administrator
- Transport cost is zero
- The network is homogeneous

Peter Deutsch's list from 1994 — every one still bites. Design for the opposite of each.

03

— Benefits & risks

What you gain

Independent delivery

Each service ships on its own schedule — a payments fix doesn't wait for catalog QA or anyone's release train.

Targeted scaling

Scale the checkout path ten-fold for the sale and leave the admin panel on one instance. Capacity follows load.

Fault isolation

A crashed recommendations service degrades the page. It does not take order-taking down with it — if you designed for it.

What it costs

The network, everywhere

Every call can fail, stall or return twice. Code that assumed a function call now needs timeouts, retries and idempotency.

Consistency gets harder

No transactions across services. Reserving stock and charging a card becomes a saga — with compensation when step two fails.

Operational surface

Every service is a pipeline, a dashboard, an alert policy and an on-call story. The platform work is real and permanent.

"Any organization that designs a system will produce a design whose structure is a copy of the organization's communication structure."

— Melvin Conway, 1968

04

— Choosing an architecture

Monolith or microservices

Modular monolith

Complex domain, one team: keep one deployable but enforce module boundaries inside it — the seams to split along later.

Microservices

Complex domain, many teams: split along bounded contexts so each team owns a service end to end.

Monolith

Simple domain, small team: the default. Distribution here is pure overhead — ship the monolith and move on.

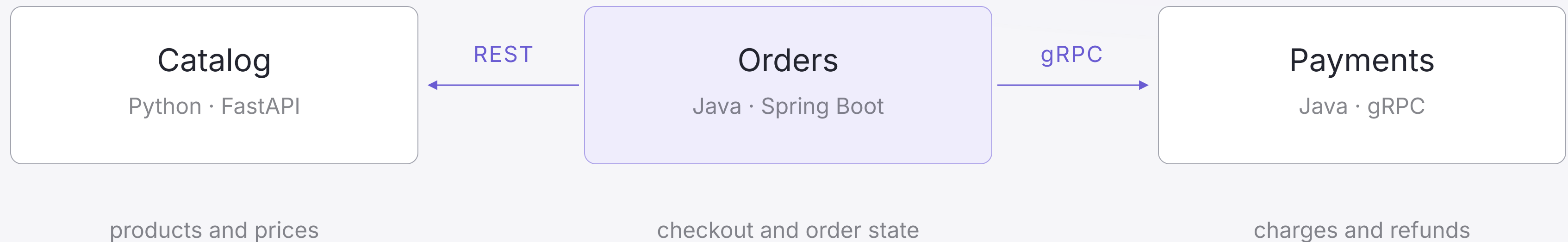
A few coarse services

Simple domain, many teams: split by team ownership, not by entity — three services, not thirty.

← fewer teams · simpler domain below

more teams · richer domain above →

The running example



Session 2 designs these two contracts; session 3 builds all three services and runs them together.

github.com/emreakis/backend-bootcamp-store

Recap

- A monolith is one deployable — distribution trades its simplicity for independence
- SOA centralizes integration in a bus; microservices push it to the endpoints
- The network introduces failure modes a monolith never sees — design for them
- Architecture follows the org chart: draw boundaries where teams are

Discussion

- Where is a system you know on the monolith–microservices spectrum?
- Which of the eight fallacies has bitten a project you worked on?
- What would you split out of that system first — and what breaks when it's down?

Next: API design

Session 2 — how services talk: REST and gRPC, and the contracts between them.

github.com/emreakis/backend-bootcamp-store